Teaching a simulated drone to hover

TD-MPC2 vs PPO on the Crazyflie hover task
Batch 1 results: 11 seeds per method

Mounir Zouhari · Supervisor: Prof. Colin Bellinger · University of Ottawa
Report date: October 13, 2026 (updated with the 1-million-step results)

What this report is

A plain-language summary of every experiment run so far: the original September run, 11 training runs of TD-MPC2 on the home PC, 11 more on the Digital Research Alliance cluster (Narval), 11 runs of a bigger TD-MPC2, 3 runs of that bigger TD-MPC2 trained 20× longer, 11 runs of PPO on the same task, and a test of the lab's existing PPO model.

No reinforcement-learning background is assumed. Section 2 explains every idea needed to read the results, and a glossary is at the end.

1. The short version

One drone, one job: take off from the floor and hover at a fixed point 1 metre up, for 10 seconds. We compared two learning methods, **TD-MPC2** and **PPO**, each trained 11 times (11 "seeds") so that luck can be told apart from real differences.

Main findings

1. TD-MPC2 learns to fly from very little practice. After 50,000 steps (about 17 minutes of simulated flying) it reliably takes off and holds the right height, but typically ends about **16–34 cm** to the side of the target (median of each 11-seed batch), slowly drifting.

2. With more practice the drift disappears. The bigger TD-MPC2 trained for 1 million steps hovers **5.7 cm** from the target on average, within 10 cm for about 90% of every flight, with no crashes from its training start (3 of 3 runs). All three reached this level after about 100,000–175,000 steps. PPO needed about **10–20 million** steps for similar precision: roughly **100× more practice**.

3. PPO is faster on the clock. Because it simulates 2,048 drones at once, PPO reaches a near-perfect hover in about 4 minutes on the same GPU; TD-MPC2 (one drone) needed about an hour to reach a similar hover. PPO's typical (median) run ends **2.8 cm** from the target, but 4 of 11 PPO runs failed or drifted.

4. Trained long enough on one route, both methods memorise it. From the 4 sideways starting points they never practised, PPO and the long-trained TD-MPC2 crash in almost every flight. The briefly trained TD-MPC2 was rougher but more robust there (39 crashes in 396 flights), probably because it had not specialised yet. The lab's PPO, trained on random starts, works from every start: practising from many positions is what makes a controller robust (Batch 2).

5. Results vary a lot between seeds. The same method, with the same settings, gave anything from a real 8 cm hover to a 2.3 m failure after 50,000 steps. A single run (like the September one) cannot show this.

6. The bigger, "official" TD-MPC2 settings are slow starters. At 50,000 steps they were worse than the small settings (about 1 m off, mostly crashing); with more practice they became the best TD-MPC2 result.

In one sentence: TD-MPC2 needs about 100× less practice than PPO to hover precisely, PPO finishes sooner on the clock thanks to massive parallel simulation, and both only learn the route they practise.

2. Background in plain language

2.1 The task

A simulated **Crazyflie**, a 27-gram palm-sized drone about 9 cm across, starts 20 cm above the floor at position (0, 0, 0.2) metres. Its goal is the point (0, 0, 1), one metre up. Every 0.02 seconds (50 times per second) the controller chooses 4 numbers: **thrust** (how hard to push up) and three small twisting forces (**roll, pitch, yaw**) that tilt or turn the drone. A flight lasts at most 500 decisions, which is 10 seconds of simulated time. The physics is computed by MuJoCo/MJX, a standard robotics simulator.

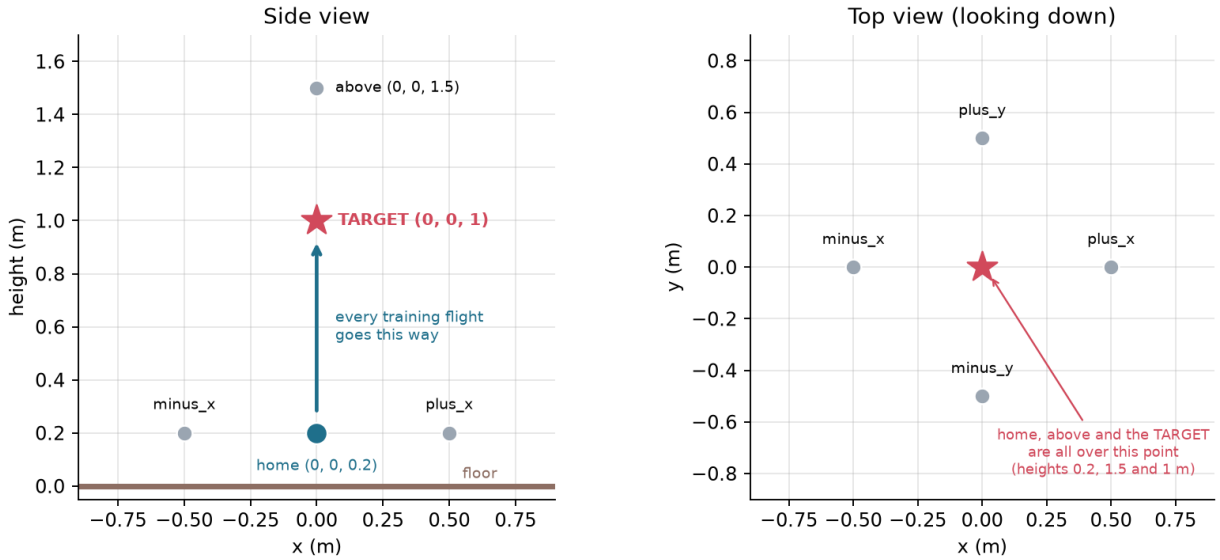


Figure 1. The task. During training every flight starts at the blue point and aims for the red star. After training, each drone is also tested from the 5 grey points it never practised from.

A flight ends early (a **crash**) if the drone drops below 5 cm, or drifts more than 2 m further from the target than where it started ("out of bounds").

2.2 What reinforcement learning (RL) is

RL is learning by trial and error, like learning to ride a bike. Nobody tells the drone which motor commands are correct. Instead it tries things, and after every 0.02 s step it receives a score called the **reward**. Over thousands of attempts it learns which behaviour collects the most reward. In this task the reward per step is:

- **+0.001** for still flying (survival bonus),
- **up to +0.01** for being close to the target (largest when exactly on it),
- **+ how much closer** it got to the target during that step (negative if it moved away),
- **−1** once, if it crashes.

The total over a flight is called the **return**. A good 10-second hover scores about +5 to +6.

2.3 The two learning methods

	TD-MPC2	PPO
Idea	Learns a **model** of how the drone moves, then **imagines** many possible short action plans before every decision and picks the best one ("model-based planning").	Learns a direct reflex: "in this situation, do this" (a **policy**), improved a little after every batch of experience. No imagination or planning.
Practice style	1 drone, learns after every step, re-uses old experience many times.	2,048 drones flying at once on the GPU; each experience is used for a few updates and then thrown away.
Strength	Learns a lot from little experience ("sample-efficient").	Each step is extremely cheap, so it can collect huge amounts of experience quickly.
Code used	JAX version of TD-MPC2 by S. Flandermeyer (faithful to the 2024 paper by Hansen et al.).	The lab's existing Brax PPO code and settings (gpu_training/ppo_training.py).

2.4 Seeds: why every experiment was run 11 times

Training involves a lot of randomness: the starting values of the neural networks, the random exploration at the beginning, sensor noise. The **seed** is a single number that fixes all of that randomness. Changing the seed gives a new, independent training run with exactly the same settings. Like training 11 students with the same course: if only one is tested, you cannot tell whether the course is good or the student was lucky. Professor Bellinger asked for 11 seeds per experiment, which is standard practice in RL research.

2.5 "Small" and "standard" TD-MPC2

The same TD-MPC2 algorithm can be run with a small or a large brain and a short or long thinking time. The September run used reduced settings to train quickly; the "standard" settings are the ones published with TD-MPC2.

Setting	Small (September, config_hover.yaml)	Standard (published defaults)
Brain size (internal summary / layer width)	128 / 128 (about 0.4 M numbers)	512 / 512 (about 5 M numbers)
Plans imagined per decision	128 plans × 3 refinement rounds	512 plans × 6 rounds
How far ahead it imagines	3 steps (0.06 s)	3 steps (0.06 s)
Experience used per learning step	64 samples	256 samples
Random flying before learning starts	500 steps	2,500 steps

3. How the drones were tested

After training, each drone flies **test flights**. It no longer learns during these; we only watch. Every trained drone flew 3 test flights from each of the 6 starting points in Figure 1, always aiming for (0, 0, 1). That is 18 test flights per training run, or 198 per group of 11 seeds. All methods were tested in exactly the same way, with the same noise.

Measurement	What it means	Good value
Late-flight error	Average distance from the target during the last 2 seconds of the flight. The main score. 0.30 m = the drone ends up hovering 30 cm away from the target.	below 0.10 m
Horizontal / vertical error	The same distance split into sideways and height parts.	small
Time within 10 cm	Share of the flight (after the first 2 s of take-off) spent within 10 cm.	near 100%
Crash	Fell below 5 cm, or flew out of bounds, before the 10 seconds were over.	no crashes
Return	Total reward collected in a flight (see 2.2).	about +5 to +6
Learning curve	Every 5,000 training steps (PPO: about every 4 M) the drone does 3 test flights, to show how it improves during training.	going down (error) / up (return)

How to read "0.32 ± 0.09 m". The first number is the **average over the 11 seeds**. The second is the **standard error**: roughly how much that average could move if we repeated the whole 11-seed experiment. Two results whose ± ranges overlap are not clearly different.

Median run. Line up the 11 results from best to worst and take the 6th. Unlike the average, one disastrous run cannot drag it far, so it is the best single answer to "what does a typical run achieve?". In the plots, the median run is drawn as a thick red line and all others as thin grey lines.

Environment steps vs. wall-clock time. An environment step is one 0.02 s decision of one drone. TD-MPC2 trained for 50,000 steps; PPO, with 2,048 drones at once, for about 79 million. Wall-clock time is the real minutes the computer needed. Comparing both is important: one method can need fewer steps but more minutes.

4. All experiments at a glance

#	Experiment	Where	Runs	Training	Purpose
A	September run (TD-MPC2 small)	home PC	1	50k steps	The original result, re-tested
B	TD-MPC2 small	home PC (RTX 3060)	11	50k steps	How much do seeds vary?
C	TD-MPC2 small, repeated	Narval cluster	11	50k steps	Same as B, on the same hardware as D and E
D	TD-MPC2 standard (big)	Narval cluster	11	50k steps	Does a bigger brain help?
E	PPO on the same task	Narval cluster	11	about 79M steps	The fair PPO comparison
F	Lab's existing PPO model	(already trained)	1	60M steps, random tasks	Reference: how good can it get?
G	TD-MPC2 standard, long	Narval cluster	3	1M steps	Does more practice fix the drift?

Narval runs used one 2/7 slice of an NVIDIA A100 GPU each. All methods were trained on exactly the same task: same start, target, reward, sensors, 10-second flights, and the same meaning of the 4 controls. The only exception is F, which the lab trained earlier on a harder version (random start and target every flight, slightly randomised drone physics, 30-second flights).

5. Results

5.1 The September run, re-tested (experiment A)

The saved September model was loaded in a fresh program and flown again. It reproduced its original result (0.30 m from the training start), which confirms the saved model and the testing code work. From the 5 new starts it never crashed, but stayed 0.18–0.38 m away from the target.

Start	Late error	Crashes
home	0.30 m	0/3
plus_x	0.38 m	0/3
minus_x	0.30 m	0/3
plus_y	0.18 m	0/3
minus_y	0.30 m	0/3
above	0.19 m	0/3

5.2 TD-MPC2 small, 11 seeds on the home PC (experiment B)

Seed	Late error	Sideways error	Height error	Time within 10 cm	Crashes (of 18 flights)
0	2.35 m	2.27 m	0.60 m	0%	12
1	0.41 m	0.41 m	0.01 m	0%	6
2	0.34 m	0.34 m	0.04 m	8%	0
3	0.27 m	0.26 m	0.07 m	2%	3
4	0.83 m	0.82 m	0.07 m	12%	0
5	0.50 m	0.40 m	0.29 m	0%	1
6	0.08 m	0.08 m	0.01 m	54%	6
7	0.42 m	0.35 m	0.20 m	10%	0
8	0.24 m	0.21 m	0.08 m	8%	0
9	0.30 m	0.29 m	0.08 m	23%	0
10	0.24 m	0.23 m	0.03 m	0%	0
All (mean ± SE)	**0.54 ± 0.19 m**	**0.51 ± 0.18 m**	**0.13 ± 0.05 m**	**11 ± 5 %**	**28/198**

What it means. The typical (median) run ended **0.34 m** from the target, close to the September run (0.30 m), so September was a typical result. But runs ranged from a real hover (seed 6, 8 cm, within 10 cm for half the flight) to a failure (seed 0, 2.3 m). Height was learned well; almost all of the error is sideways.

Why did seed 0 fail?

Seed 0 learned to fly well: at 45,000 steps its test flights were 0.27 m from the target with no crashes, better than September. In the last 5,000 steps it suddenly got worse and its final test flights drifted out of bounds. This was checked with two independent testing methods (the new one and the original September one), which agree. This kind of late "collapse" is a known instability of RL training. It is a real result, not a bug, and it is kept in the averages.

Seed 0 of September and seed 0 today ended differently even with the same seed number: GPU arithmetic is not bit-for-bit repeatable, and over 50,000 learning steps tiny differences snowball.

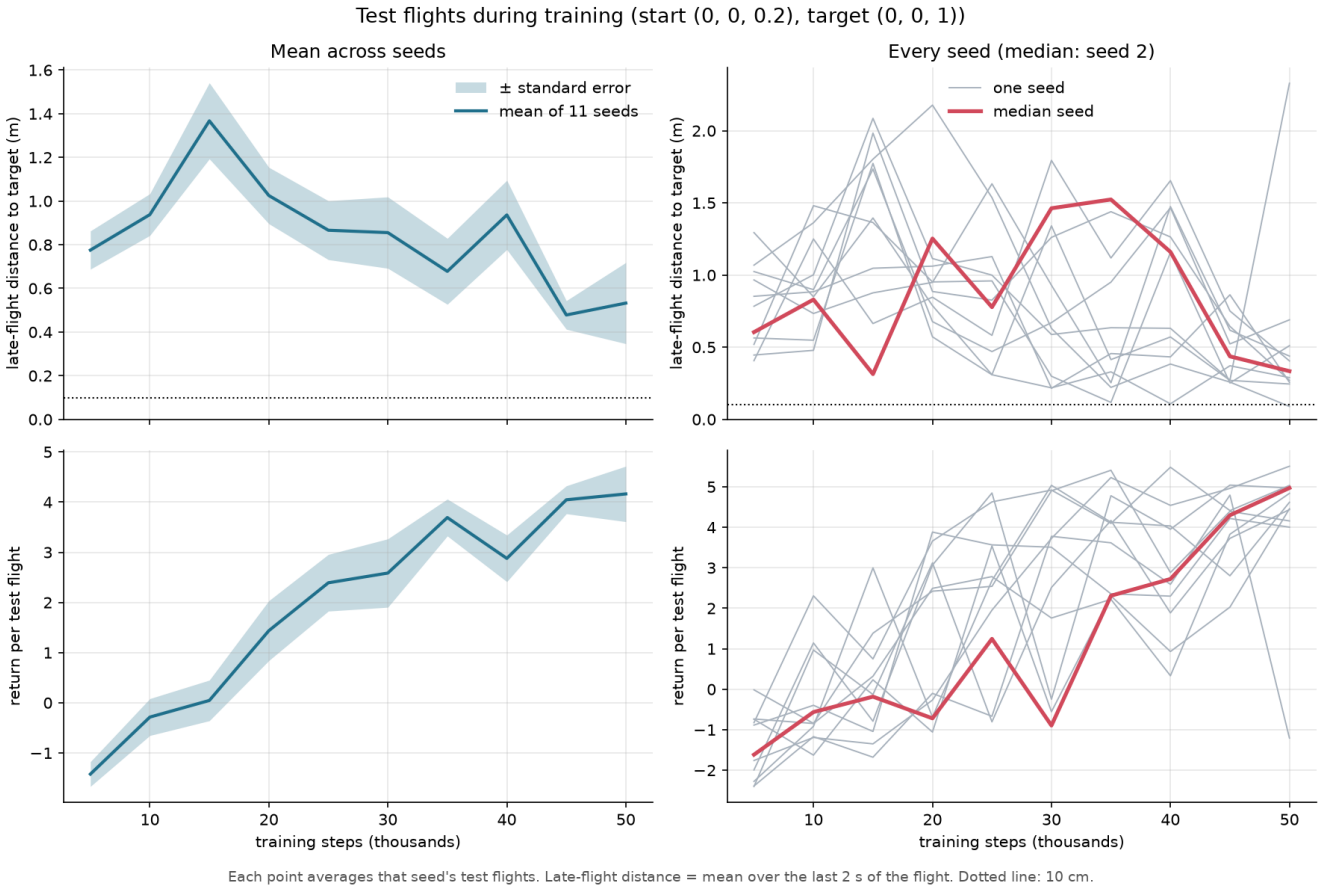


Figure 2. Experiment B during training. Left: average of 11 seeds with its standard-error band. Right: every seed (grey) and the median seed (red). Top: distance to target in test flights (lower is better). Bottom: return (higher is better). The return was still rising at 50,000 steps, so training was not finished.

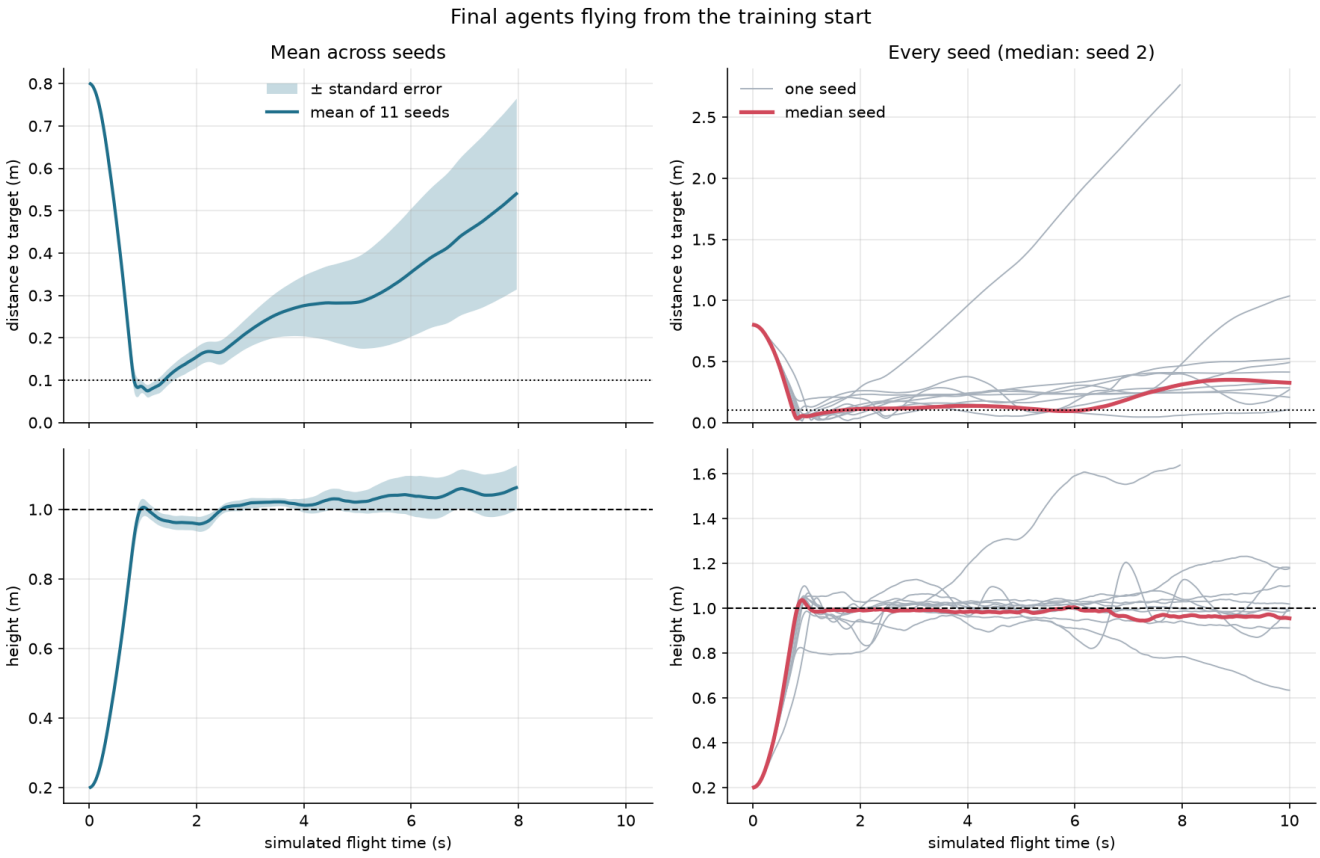


Figure 3. Experiment B: the final drones flying from the training start. All reach 1 m height within about 1 second (bottom). Then the distance to the target slowly grows (top): the sideways drift, about 3 cm per second. Each line stops when one of its flights crashes.

5.3 The same experiment repeated on the cluster (experiment C)

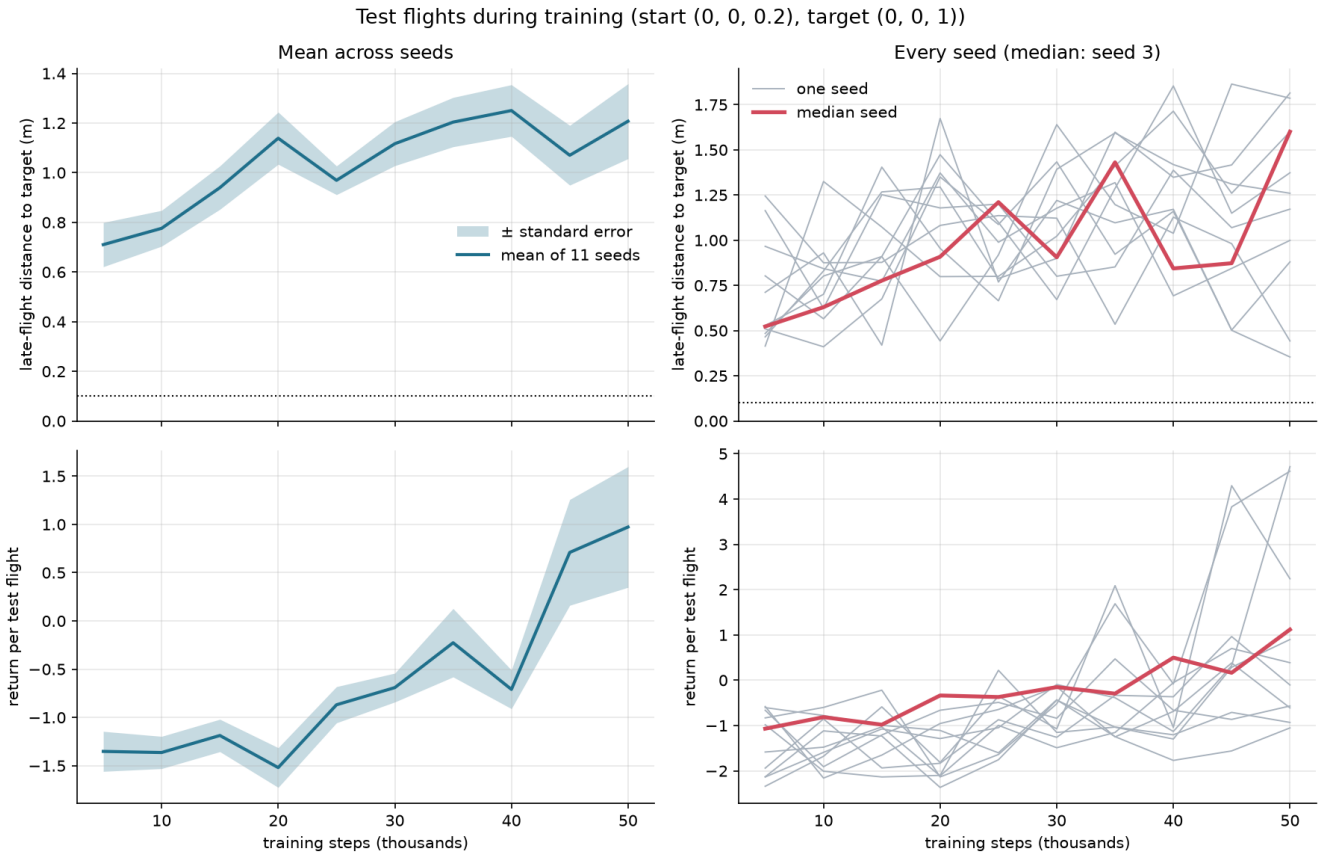
Experiment B was repeated on Narval with identical code and settings. Result: **0.32 ± 0.09 m** (median 0.16 m, 11 crashes in 198 flights), compared with 0.54 ± 0.19 m on the home PC. The ± ranges overlap, so the two batches are consistent: the same method with different luck. Together they are **22 independent runs** with a median of 0.28 m. Experiment C is used in the comparisons below because it ran on the same hardware as D and E, which makes training times comparable.

5.4 TD-MPC2 with standard (big) settings (experiment D)

The bigger version did clearly worse after 50,000 steps: **1.06 ± 0.14 m** from the training start and 182 crashes in 198 flights. It was still improving: its return rose steadily during training (Figure 4, bottom). The likely reason is that larger networks need more data before they pay off. Note that the standard settings change about 10 things at once (brain size, number of plans, batch size, and so on), so we cannot yet say which one matters.

Careful: why the distance curve goes up while the drone improves

Early in training the drones crash right after take-off, close to the start, about 0.8 m from the target. Later they fly much longer before failing and drift far away first. Because the late-flight error is measured over the last 2 seconds before the flight ends, it gets larger even though the drone is improving. For runs that crash a lot, look at the return and the crash count instead.



Each point averages that seed's test flights. Late-flight distance = mean over the last 2 s of the flight. Dotted line: 10 cm.

Figure 4. Experiment D during training. The return (bottom) keeps rising, but after 50,000 steps most flights still end in a crash.

5.5 PPO on exactly the same task (experiment E)

PPO used the lab's own settings and code; only the task was changed to match TD-MPC2. From the training start it was the most precise: median **2.8 cm**, and 7 of 11 seeds stayed within 10 cm for the whole flight. But 4 of 11 seeds failed or drifted from the training start, and from the new starting points PPO crashed in most flights (see 5.7).

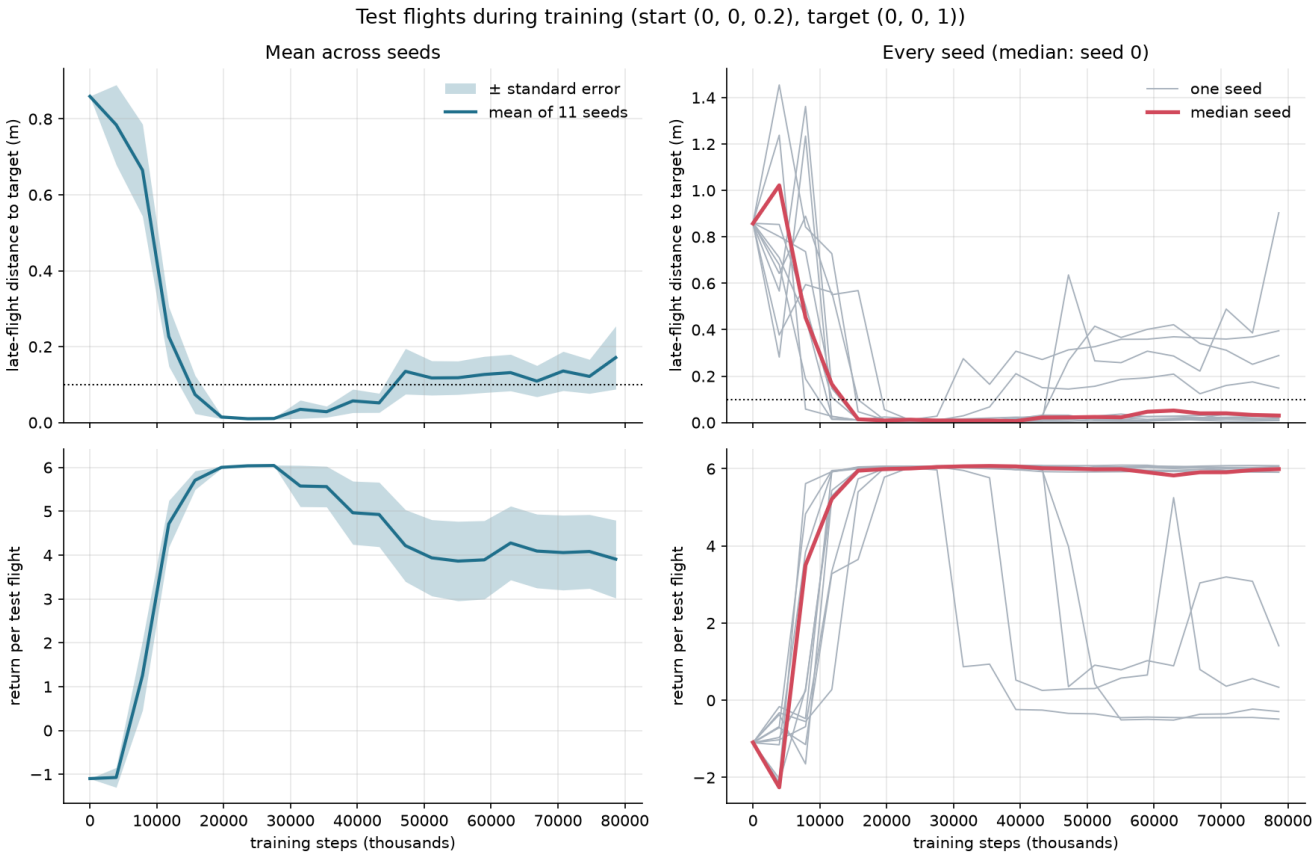


Figure 5. Experiment E during training (x axis in thousands of steps, so the end is about 79 million). PPO reaches an almost perfect hover around 20–30 million steps, then gets somewhat worse again towards the end: its final model is not its best one.

5.6 The lab's existing PPO model (experiment F)

This model was trained earlier by the lab on a harder version of the task (random start and target every flight), for 60 million steps according to the lab notebook's settings. Flown through our exact test, it was excellent from every start:

Start	Late error	Crashes
home	6.4 cm	0/3
plus_x	3.5 cm	0/3
minus_x	8.8 cm	0/3
plus_y	5.9 cm	0/3
minus_y	8.2 cm	0/3
above	20.9 cm	0/3

This shows two things: the hover task is clearly learnable to a few centimetres, and practising from many different positions produces a controller that works from many positions. That is exactly what the next experiment (Batch 2, random starts and targets) will test for both methods.

5.7 Head-to-head comparison (experiments C, D, E)

	TD-MPC2 small	TD-MPC2 standard	PPO
Training steps	50,000	50,000	about 79 million
Training time per seed (same GPU)	14.0 ± 0.3 min	20.2 ± 0.2 min	11.4 ± 0.0 min
Late error, training start	0.32 ± 0.09 m	1.06 ± 0.14 m	0.17 ± 0.08 m
Median run, training start	0.16 m	1.02 m	0.03 m
Late error, average of all 6 starts	0.45 ± 0.12 m	1.12 ± 0.05 m	0.71 ± 0.03 m
Crashes, all 198 test flights	11 (6%)	182 (92%)	157 (79%)

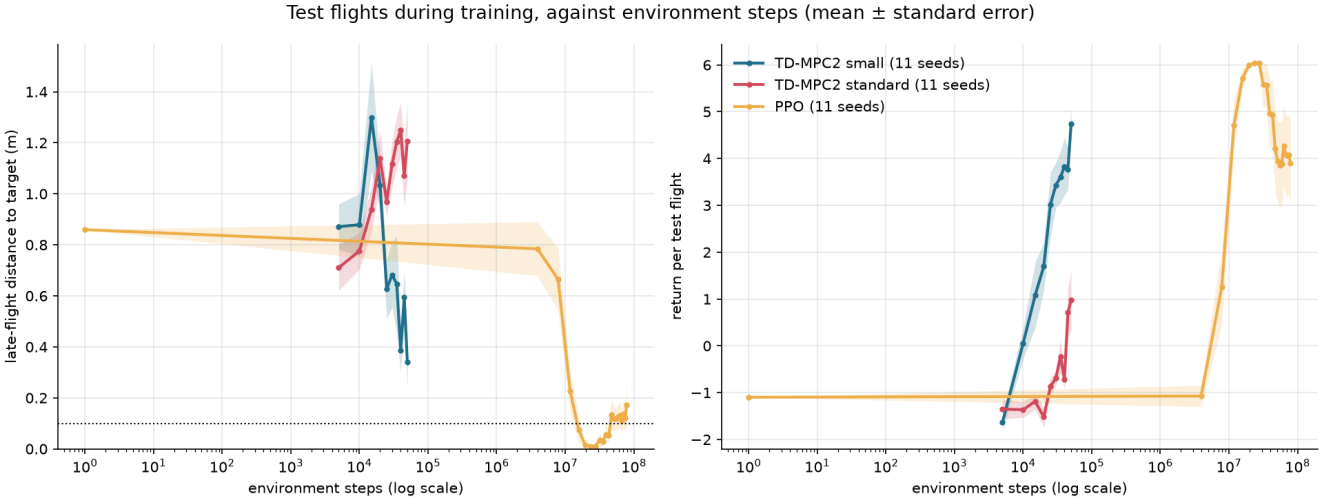


Figure 6. Learning against **practice** (environment steps, log scale: each grid line is 10× more). TD-MPC2 (blue) learns within 50,000 steps. PPO (orange) shows no progress until about 10 million steps. **TD-MPC2 is roughly 200× more sample-efficient.**

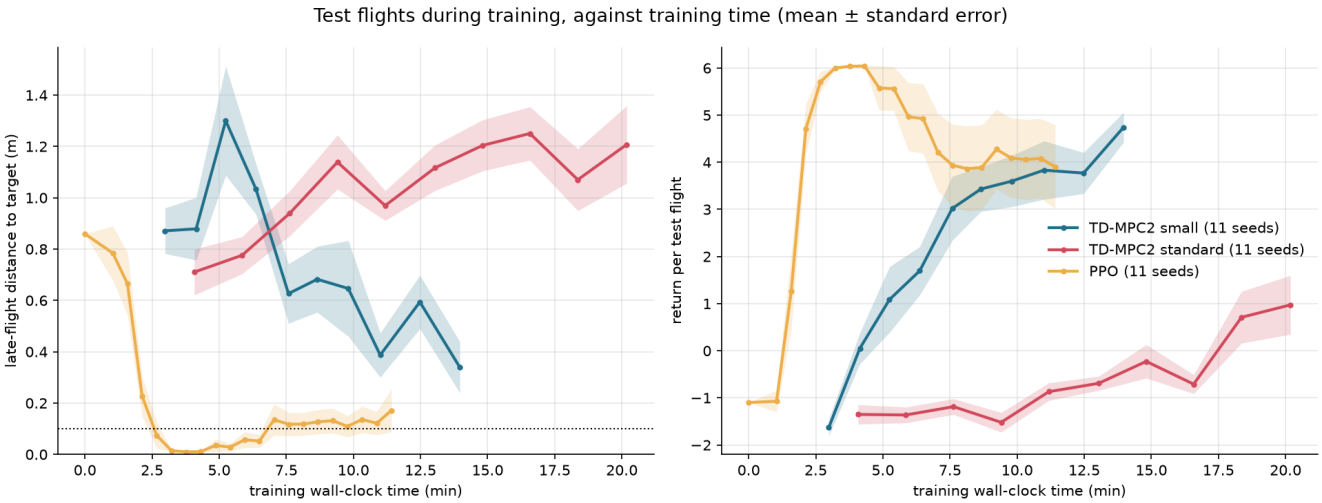


Figure 7. Learning against **real time**, all on the same GPU. PPO reaches its best hover within about 4 minutes; TD-MPC2 small is still about 30 cm off after 14 minutes. **On the clock, PPO wins**, because it simulates 2,048 drones at once while TD-MPC2 flies one.

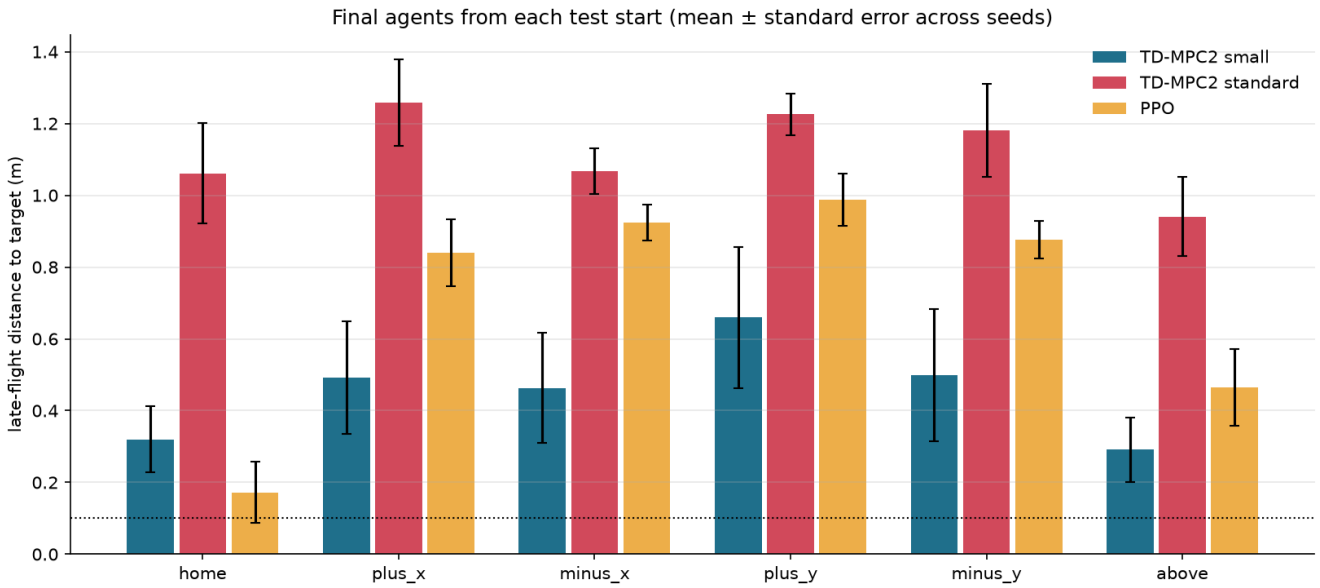


Figure 8. Final drones from each starting point (lower is better, dotted line = 10 cm). PPO is best from the training start ("home") but much worse from the 5 new starts. TD-MPC2 small is the most consistent.

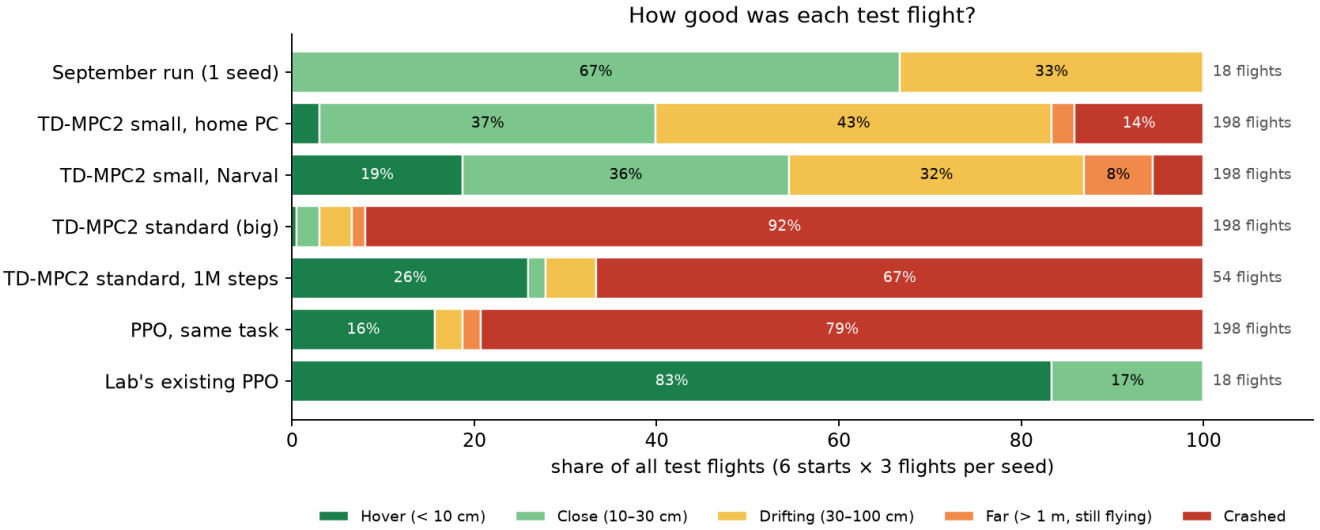


Figure 9. Every test flight (all 6 starts) sorted into quality groups by its late-flight error. Green = precise, red = crashed. TD-MPC2 small after 50,000 steps rarely crashes but is rarely precise. PPO and the 1-million-step TD-MPC2 (section 5.8) are precise from the training start but crash from most new starts.

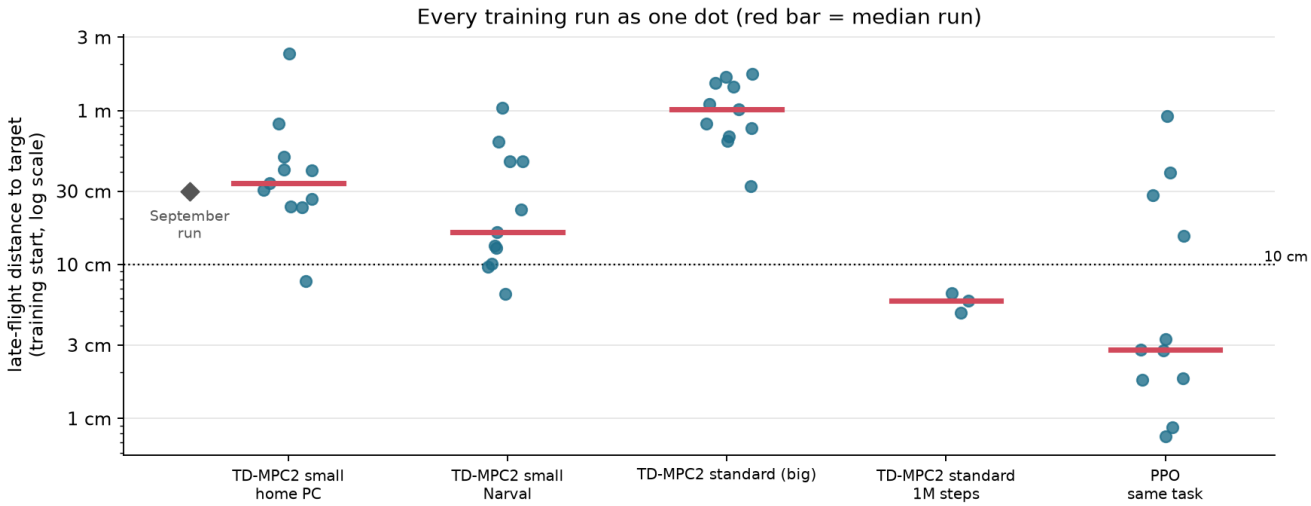


Figure 10. Each dot is one complete training run, scored from the training start. The spread inside each column shows how much luck matters. Note the log scale: each labelled line is about 3× the one below.

5.8 Longer training: 1 million steps (experiment G)

Question: does the sideways drift come from too little practice? To test this, the standard (big) TD-MPC2 was trained 20× longer: 1,000,000 steps instead of 50,000, with 3 seeds. Only two settings changed besides the length: the replay memory was enlarged to 1 million steps (the published value, so nothing is forgotten) and the model was saved less often. Each run took about 6 hours on a GPU slice.

Seed	Late error (training start)	Height error	Time within 10 cm	Crashes, training start	Crashes, all 18 flights
0	4.9 cm	0.8 cm	89%	0/3	12/18
1	6.5 cm	0.6 cm	97%	0/3	12/18
2	5.8 cm	1.4 cm	90%	0/3	12/18
All (mean ± SE)	**5.7 ± 0.5 cm**	**0.9 ± 0.2 cm**	**92 ± 3 %**	**0/9**	**36/54**

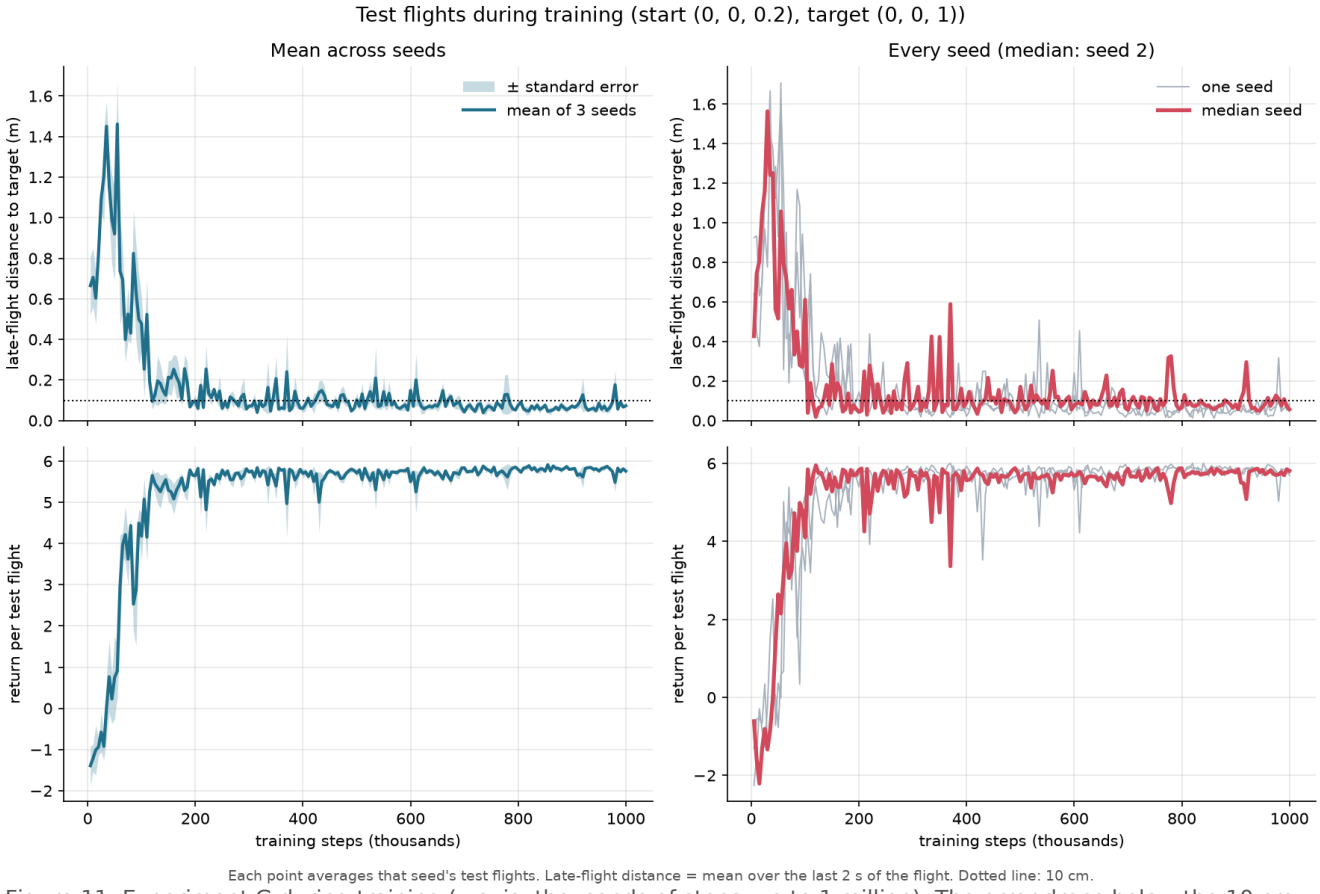


Figure 11. Experiment G during training (x axis: thousands of steps, up to 1 million). The error drops below the 10 cm line after about 100,000–175,000 steps and stays there, with the return close to its maximum (about 6). Occasional single test points jump to 20–60 cm, but the runs recover.

So is the drift fixed?

From the training start: yes. All 3 runs hover 5–6 cm from the target, keep the height within about 1 cm, spend about 90% of the flight within 10 cm, and never crash. This is more consistent than PPO (whose median run is more precise, but 4 of 11 runs failed). So the drift seen after 50,000 steps was mainly a lack of practice.

From the sideways starts: no, it got worse. From the 4 starts 0.5 m to the side, all 36 test flights crashed: sometimes the drone never takes off (it falls within half a second), otherwise it climbs, never gets closer than about 20–60 cm, and loses control after a few seconds. From "above" it still reaches about 23 cm without crashing.

Start	Late error	Crashes
home	0.057 ± 0.005 m	0/9
plus_x	1.027 ± 0.030 m	9/9
minus_x	1.048 ± 0.065 m	9/9
plus_y	0.978 ± 0.019 m	9/9
minus_y	1.385 ± 0.227 m	9/9
above	0.233 ± 0.106 m	0/9

What this changes. After 50,000 steps the small TD-MPC2 looked much more robust to new starts than PPO. This experiment suggests that was mostly because it had not specialised yet: once TD-MPC2 is trained long on a single route, it memorises that route just like PPO did. A fair test of robustness therefore needs training from many starting points, which is Batch 2.

A detail in the environment that may matter. The drone's sensors report the direction to the target divided by the distance at the start of the flight (0.8 m from the training start, about 0.94 m from the sideways starts). So the same physical situation produces slightly different numbers depending on where the flight began, which makes a single-start controller even less likely to transfer. In Batch 2 this varies all the time during training, so the controller will learn to handle it.

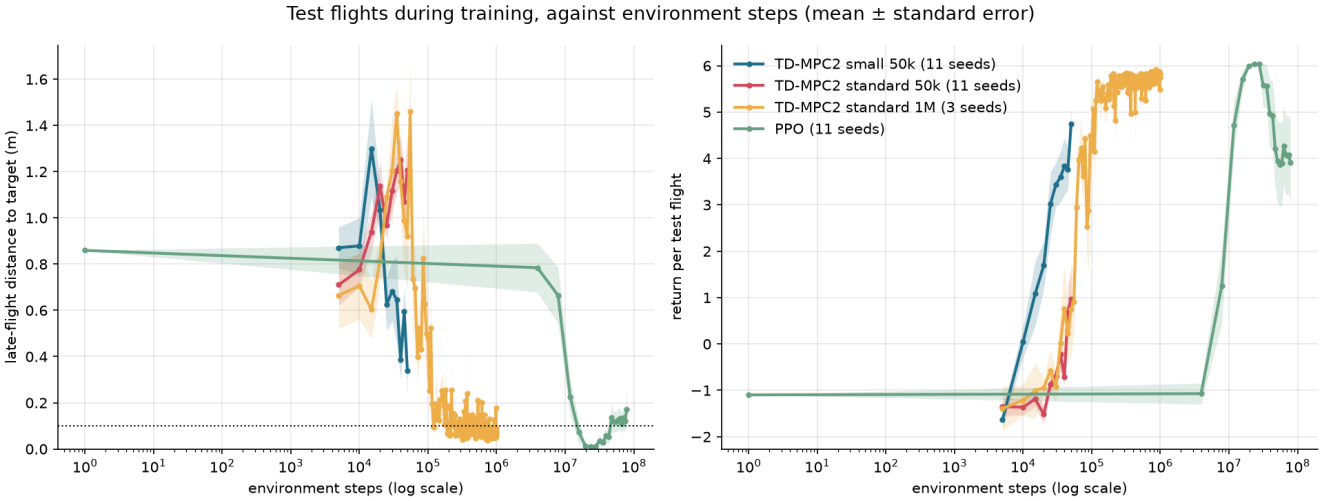


Figure 12. All methods against practice (log scale). The 1-million-step TD-MPC2 (orange) reaches PPO's level of return (about 6) after roughly 200,000 steps; PPO (green) needs about 20 million.

6. What it all means

6.1 What a typical TD-MPC2 flight looks like after 50,000 steps

- Take-off to 1 m in about 0.8 seconds, a small overshoot to about 1.08 m, then the height is held within a few centimetres.
- Meanwhile the drone slides slowly sideways, about 3 cm per second, and after 10 seconds it is typically about 30 cm from the target.
- Each run drifts in its own direction, all around the compass. A problem in the simulator would push every run the same way, so this is a learning limitation, not a physics bug.
- Crashes are uncommon and happen late (median 7 seconds into the flight), after the drift has built up: in experiment B, 16 by falling and 12 by flying out of bounds.

6.2 Why does it drift sideways?

A drone cannot push sideways directly. To move sideways it must tilt, accelerate, tilt back to brake, and level out exactly on the target: a slow chain that takes about a second. Nothing pulls it back on its own, so any small tilt makes the error grow over time. Height is much easier: more thrust immediately means up. Likely reasons TD-MPC2 has not mastered the sideways part yet:

- **The reward barely cares about 30 cm.** Being exactly on target earns 0.0100 per step; being 30 cm away earns 0.0071. That difference is tiny compared with the −1 crash penalty: there is a strong reason not to crash, but only a weak reason to be precise.
- **Little practice at correcting.** Every training flight starts in the same place, so the drone rarely experiences being off to the side and coming back. 50,000 steps is the equivalent of only about 100 full 10-second flights.
- **Short look-ahead.** The planner imagines only 0.06 s ahead and relies on its learned estimate of the future beyond that, which is still inaccurate after 50,000 steps.
- The task itself is learnable: PPO reaches 1–3 cm from the training start, and the lab's PPO from all starts.

What experiment G showed. With 20× more practice (1 million steps) the drift from the training start disappeared: about 5–6 cm, no crashes. So "little practice at correcting" was the main reason. The weak reward for precision did not prevent precise hovering once the drone had enough experience.

6.3 Answers to the original questions

Question	Answer so far
Does TD-MPC2 learn to hover?	Yes. After 50,000 steps it takes off and holds height but typically drifts 15–35 cm sideways. After about 150,000 steps (bigger settings, 3 runs) it hovers within about 5–6 cm with no crashes.
Was the September result reliable?	It was a typical 50,000-step run. But seeds range from 8 cm to 2.3 m, so one run alone hides most of the picture.
Does it work from new starting points?	Only roughly, and only while training is short. After 50,000 steps it flies from new starts about as well as from its own; after 1 million steps it has specialised and crashes from the sideways starts. Training from many starts (Batch 2) is needed.
How does it compare with PPO?	About 100× less practice needed for a precise hover, and more consistent between runs; but slower in real time on this GPU, because PPO simulates 2,048 drones at once. Both memorise the single route they are trained on.
Do the bigger, official settings help?	Not at 50,000 steps (worse than the small settings), but with more practice they gave the best TD-MPC2 result. The small settings have not yet been trained longer, so a fair small-vs-big comparison at long training is still open.

7. Limitations to keep in mind

- **Few flights per seed.** Each seed was tested with 3 flights per starting point. Averages over 11 seeds are reasonably stable; single-seed numbers less so.
- **Experiment G has only 3 seeds.** The 1-million-step result is very consistent across those 3, but it should be repeated with 11 seeds (and for the small settings too) before it is reported as final.
- **Final model only.** Scores use the model at the very end of training. Both TD-MPC2 (seed 0) and PPO (most seeds) were better a little earlier, so final scores include the effect of late instability.
- **Late-flight error and crashes.** For a crashed flight it measures where the drone was just before failing, which is not comparable with a flight that survived. Read it together with the crash count.
- **Not exactly repeatable.** GPU arithmetic is not bit-for-bit deterministic, so re-running a seed gives a similar, not identical, result. That is why groups of 11 seeds are compared.
- **Training budgets differ by design.** TD-MPC2 trained for 50,000 steps and PPO for about 79 million (the lab's setting of 60 million, rounded up by the PPO library). This is why both a steps comparison and a time comparison are shown.
- **Times.** Only experiments C, D and E ran on identical hardware. Experiment B ran two seeds at a time on the home GPU, so its 26 minutes per seed is not comparable.
- **Implementation details.** The JAX TD-MPC2 differs from the official PyTorch version in small ways (for example how the final action is picked and some optimiser details). These are the same in every TD-MPC2 run, so they do not affect the small-vs-standard comparison.

8. Next steps

- **Confirm experiment G with 11 seeds.** Since the hover is reached after about 150,000 steps, about 200,000 steps per run is enough (about 1.2 hours each on a GPU slice), for both the small and the standard settings.
- **Batch 2 (Professor Bellinger's next request).** Random start and target in every training flight, again 11 seeds per method. Both methods memorised their single training route, so this is the experiment that tests whether they can learn to fly from anywhere, as the lab's PPO does.
- **Training length in Batch 2.** Flying from anywhere is harder than one route, so Batch 2 will likely need more steps per run than 50,000; the learning curves will show when each method levels off.
- **The research question: scaling.** PPO wins on the clock because it simulates thousands of drones at once. What happens when TD-MPC2 is also given many parallel simulations? That is the next big step of the project.

Glossary

Term	Meaning
Agent / brain	The learning controller: the neural networks that decide the drone's actions.
Action	The 4 numbers chosen every 0.02 s: thrust, roll, pitch, yaw.
Observation	The 31 numbers the drone "sees": position, speed, tilt, direction to the target, last action (with a little sensor noise).
Reward / return	Score for one step / total score of a flight.
Environment step	One decision of one drone (0.02 s of simulated time).
Training run	One complete training from scratch, ending with one trained model.
Seed	The number that fixes all randomness of a training run.
Policy	A rule that maps what the drone sees to what it does.
World model	TD-MPC2's learned prediction of what will happen after an action.
Planning / MPC	Imagining many action sequences with the world model and picking the best.
PPO	Proximal Policy Optimization: a widely used RL method that improves a policy directly from large batches of experience.
Sample efficiency	How much a method learns per step of experience.
Wall-clock time	Real time the computer needed to train.
Mean ± standard error	Average over seeds ± how precisely that average is known.
Median	The middle value; not pulled around by one extreme run.
Checkpoint	A saved copy of a trained model.
Overfitting / memorising	Doing well only on exactly what was practised.
GPU slice	A share (here 2/7) of one A100 graphics card on the Narval cluster.

Result files: outputs/batch1_fixed_start, batch1_small_narval, batch1_standard, ppo_fixed_start, ppo_existing_checkpoint and comparison_batch1 in the project folder. Each analysis folder contains a summary.md with the exact numbers.